

ADVANCED MATHEMATICS

Chapter 9

Copyright © 2017 STEM Course Prep



POLYNOMIALS

- **Polynomials** are curves
 - **Degree** or **order** of equation used to describe polynomial
 - 0th order: a single value
 - $y=13.75$ $y=8.9$
 - 1st order: a linear function
 - $y=2x$ $y=5.7x+6$ $y=-8.4x-13$
 - 2nd order: a quadratic function
 - $y=3x^2+2$ $y=-9x^2+7$
 - 3rd order: a cubic equation
 - $y=8x^3$ $y=1.5x^3+2x^2+14x+1$
 - nth order: equation where n is the largest exponent
- Polynomials are represented by row vectors of coefficients
 - $y=1.5x^3+2x^2+14x+1 \longrightarrow p=[1.5 \ 2 \ 14 \ 1]$
 - $y=4x^4-2x^3+5x-1 \longrightarrow p=[4 \ -2 \ 0 \ 5 \ -1]$

POLYNOMIAL FUNCTIONS³

Find the roots and evaluate: $y=15x.^4+13x.^3+2x.^2+9x+1$

- **polyval(p, x)**
evaluates a
polynomial p at x
- **roots(p)** finds the
roots of an equation
(sets polynomial =0
and then solves for
roots)

```
Command Window
>> x=0:5;
>> y=15*x.^4+13*x.^3+2*x.^2+9*x+1

y =

         1         40        371        1612        4741        11096

>> p=[15 13 2 9 1];
>> polyval(p,x)

ans =

         1         40        371        1612        4741        11096

>> roots(p)

ans =

-1.156741380500675
 0.201103461857398 + 0.688136982785030i
 0.201103461857398 - 0.688136982785030i
-0.112132209880790

fx >>
```

CURVE FITTING

- Consider experimental data in tabular form:

Time	Velocity
0	5
2	9
4	13
6	17
8	21

- Data is typically collected based on some parameter
 - Time (as shown above)
 - Temperature
 - Pressure
 - Distance
 - Etc
- It is impossible to collect ALL data points
- What if we need data that is not collected during the experiment
 - For example, what's the velocity at time=3?

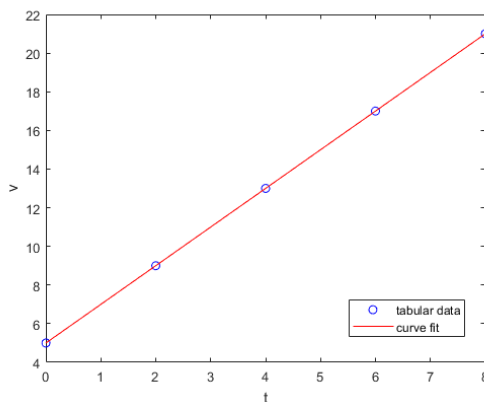
CURVE FITTING

- A way to find these missing data points is to use:
Curve Fitting
- Curve fitting is simply finding an equation that best models the data you have
 - Equation can then be used to fill in missing data points when needed
- MATLAB has built-in functions and a Curve Fitting Toolbox for this task

CURVE FITTING

- **polyfit(x,y,n)** fits an n^{th} -order polynomial to the x and y data points
 - returns a row vector of coefficients for the n^{th} -order polynomial

Time	Velocity
0	5
2	9
4	13
6	17
8	21



```
t=0:2:8;
v=5:4:21;
c=polyfit(t,v,1)
```

```
>>
c =
```

2 5

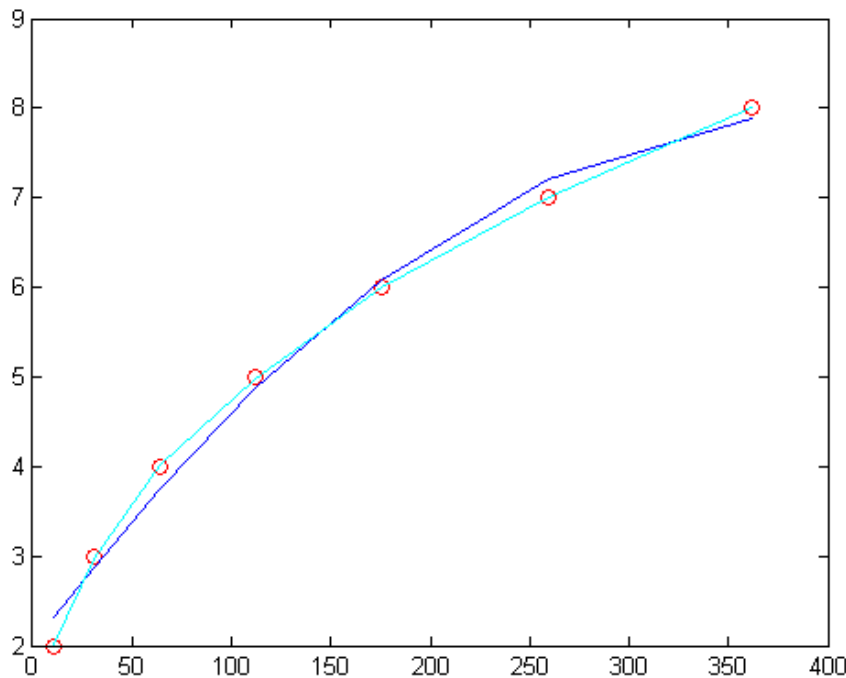
$$v(t) = 2t + 5$$

Note: Plug t values into above equation to get red line on plot

CURVE FITTING⁷

Curve fit the test data given:

x	y
11	2
31	3
64	4
112	5
176	6
259	7
362	8



```
x=[11 31 64 112 176 259 362];  
y=2:8;  
plot(x,y, 'ro')  
hold on
```

```
coeffs=polyfit(x,y,2)  
y_fit2=polyval(coeffs,x)  
plot(x,y_fit2)
```

```
coeffs=polyfit(x,y,5)  
y_fit5=polyval(coeffs,x)  
plot(x,y_fit5, 'c-')
```

INTERPOLATION AND EXTRAPOLATION

- Using the curve that has been fit through the data points, we can estimate values other than at the sampled data points
- **Interpolation** is estimating the values in between recorded data points
- **Extrapolation** is estimating outside of the bounds of the recorded data
 - both of these can be accomplished using **polyval**
- Important .. The better the curve fit, the more exact these interpolated and extrapolated values will be

INTERPOLATION: POLYVAL⁹

Scenario: Find 'missing' data at given points

Steps:

1. Use `polyfit` to find a curve that fits the data
 2. Find new data at the desired points using `polyval`
- Example: Temperature and pressure data is given from the steam tables.
 - We need to find the pressures at the following temperatures: 113 114.7 117.34

Temp	Pressure
112	1.35
114	1.429
116	1.512
118	1.6
124	1.692

→ Given data

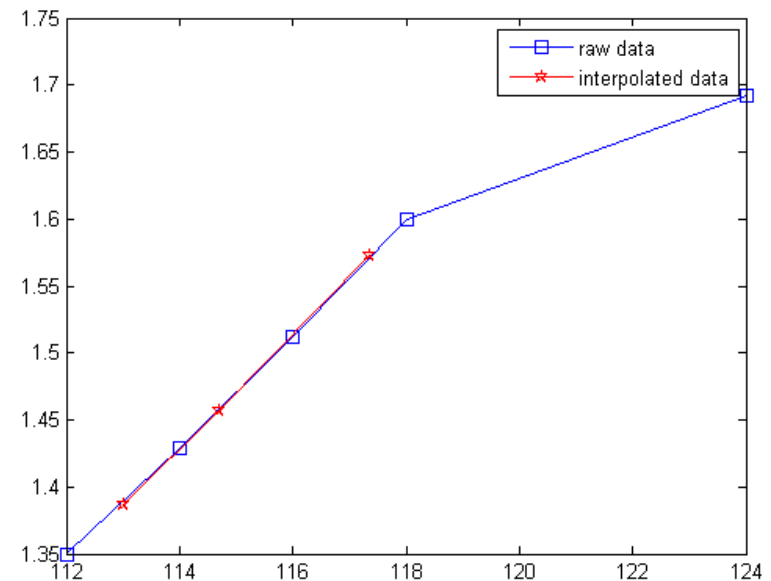
INTERPOLATION: POLYVAL

- Use `polyfit` to find a curve that fits the data
- Find new data at the desired points using `polyval`

```
x=[112 114 116 118 124];
y=[1.35 1.429 1.512 1.6 1.692];
coeffs=polyfit(x,y,3);
```

```
xi=[113 114.7 117.34];
y_interp=polyval(coeffs,xi)
```

```
plot(x,y,'bs-')
hold on
plot(xi,y_interp,'rp-')
legend('raw data','interpolated data')
```



INTERPOLATION: INTERP¹¹1

- `interp1` – built-in interpolation function
 - `interp1(x,y,xi,'method')`
 - `x,y` – given data points
 - `xi` – desired points
 - `method`- 'linear' or 'spline'
 - Linear – linear interpolation
 - Spline – cubic spline interpolation – for non-linear functions
- Results given: 'missing' y-values for the given points
 - This is a one-step interpolation process
 - `polyfit/polyval` interpolation has two steps: do a curve fit and then evaluate the data points

INTERPOLATION: INTERP¹²1

- Example: Temperature and pressure data is given from the steam tables.
- We need to find the pressures at the following temperatures: 113 114.7 117.34

Temp	Pressure
112	1.35
114	1.429
116	1.512
118	1.6
124	1.692

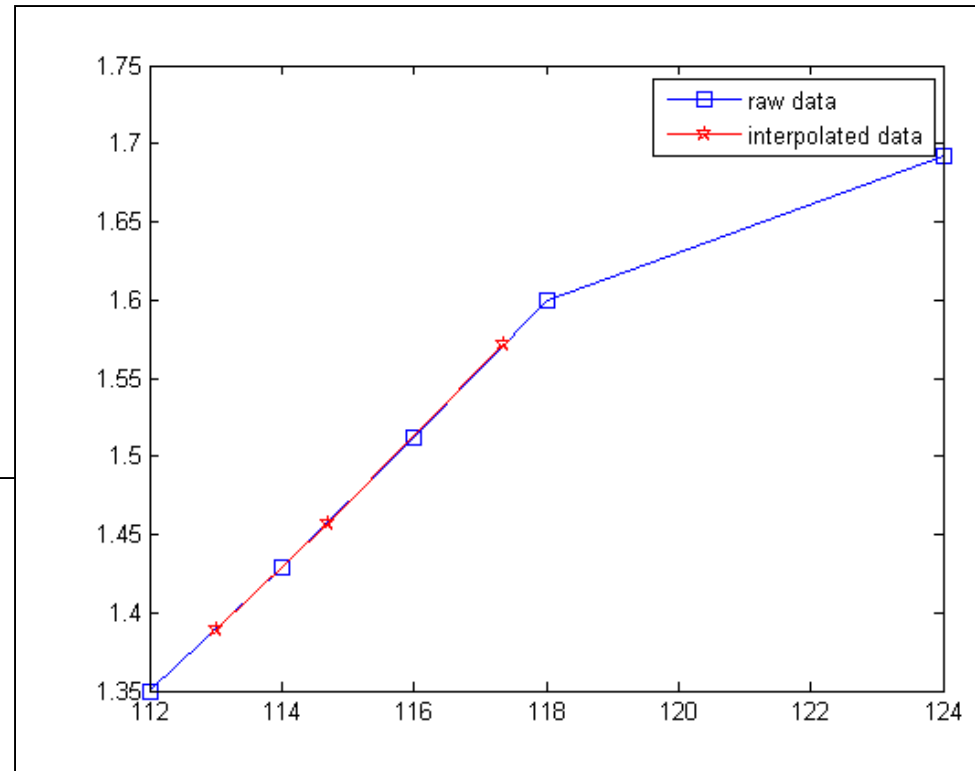


Given data

INTERPOLATION: INTERP¹³1

```
y_interp =  
1.389363636363636  
1.457297181818182  
1.571295603000000
```

```
x=[112 114 116 118 124];  
y=[1.35 1.429 1.512 1.6 1.692];  
  
xi=[113 114.7 117.34];  
y_interp=interp1(x,y,xi,'spline');  
plot(x,y,'bs-')  
hold on  
plot(xi,y_interp,'rp-')  
legend('raw data','interpolated data')
```



COMPLEX NUMBERS

- **Complex numbers** are generally written in the form:

$$z = a + bi$$

Real part

Imaginary part

$i = \sqrt{-1}$

- i (and j) are built-in constants in MATLAB
 - MATLAB defaults to i for complex numbers

```
>> z=2+3j
z =
    2.0000 + 3.0000i
```

COMPLEX NUMBER FUNCTIONS

- **complex** will return a complex number, given the real and imaginary parts
 - $c = \text{complex}(A, B)$ returns $A + Bi$
- **real** returns the real part of a complex number
 - $\text{real}(z)$ returns A if $z = A + Bi$
- **imag** returns the imaginary part
 - $\text{imag}(z)$ returns B if $z = A + Bi$
- **conj** returns the complex conjugate of a complex number
 - $\text{conj}(z)$ returns $A - Bi$ if $z = A + Bi$

DISPLAYING COMPLEX NUMBERS

- To print a complex number:
 - **disp** will display the entire complex number

```
>> z=2+3j
>> disp(z)
    2.0000 + 3.0000i
```

- **fprintf** by default will display only the real part

```
>> fprintf('%f\n',z)
2.000000
>> fprintf('%f + %fi\n',real(z),imag(z))
2.000000 + 3.000000i
```

- **isreal** – built-in function – returns 1 if variable is real, 0 if variable is complex `isreal(z) = 0`

SYMBOLIC EXPRESSIONS

- Why do we need this?
 - Allows math to be done on symbols, instead of numbers
 - Good for simplifying complex equations etc.
 - Allows normal operators to be used: +, -, * etc.
- **sym** function is used to create a symbol

```
>> b=sym('b');  
>> b*b  
  
ans =  
  
b^2  
  
>> c=3*b+4*b  
  
c =  
  
7*b  
  
>> symeqn=sym('x^4-6');  
>> 2*symeqn  
  
ans =  
  
2*x^4 - 12
```

SYMBOLIC EXPRESSIONS

- **syms** is a shortcut to declare symbols
 - `syms a b c`
is equivalent to
`a=sym('a')`
`b=sym('b')`
`c=sym('c')`
- **sym2poly** and **poly2sym** convert between symbols and polynomials

```
>> coeffs=[2 4 6];
>> poly=poly2sym(coeffs)

poly =

2*x^2 + 4*x + 6

>> d=3*poly

d =

6*x^2 + 12*x + 18

>> sym2poly(d)

ans =

        6        12        18
```

SYMBOLIC FUNCTIONS

simplify - simplifies expressions

```
>> syms x
>> func=x^3+x^2

func =

x^3 + x^2

>> simplify(func)

ans =

x^2*(x + 1)
```

collect - collects like terms

```
>> syms x y
>> collect(y^4+2*y^5+3*y^4-x)

ans =

2*y^5 + 4*y^4 - x

>> collect(x*exp(x)+3*x*exp(x)-y)

ans =

(4*exp(x))*x - y
```

SYMBOLIC FUNCTIONS

expand - multiplies out terms

```
>> syms x y
>> expand((x-2*y)*(3*exp(x)*cos(x)-y))

ans =

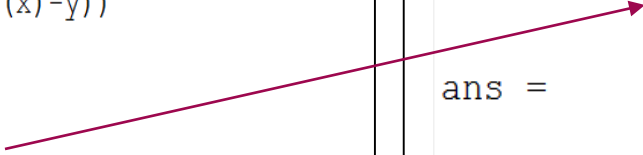
2*y^2 - x*y + 3*x*exp(x)*cos(x) - 6*y*exp(x)*cos(x)
```

factor - factors a symbolic expression

```
>> factor(ans)

ans =

-(y - 3*exp(x)*cos(x))*(x - 2*y)
```



SYMBOLIC FUNCTIONS

subs - substitutes value into expression

```
>> func=x^3+2.5*x^2+1;
>> subs(func,3)
```

```
ans =

    50.5000
```

```
>> func=3*x*sin(y)-x;
>> subs(func,'x',5)
```

```
ans =

15*sin(y) - 5
```

Specifies that 5 should be substituted for x

numden - returns numerator and denominator of a fraction

```
>> g=k*(s+1)/(s+2)*(s-3);
>> [num,den]=numden(g)
```

```
num =
```

```
k*(s + 1)*(s - 3)
```

```
den =
```

```
s + 2
```

```
>> g=k*(s+1)/((s+2)*(s-3));
>> [num,den]=numden(g)
```

```
num =
```

```
k*(s + 1)
```

```
den =
```

```
(s + 2)*(s - 3)
```

SYMBOLIC FUNCTIONS

pretty – displays 'pretty' form of equation

```
>> pretty(x^8+y^2)
```

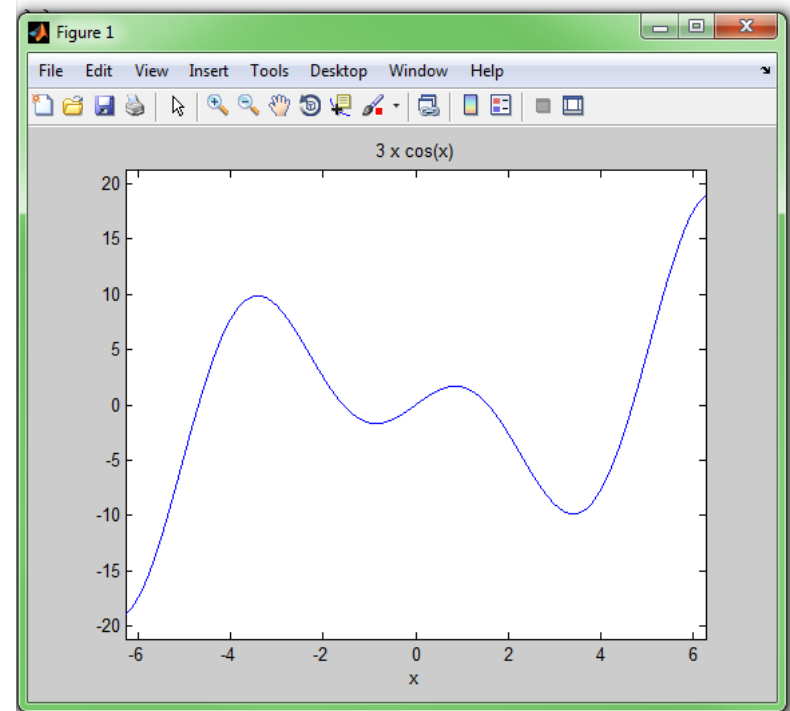
$$x^8 + y^2$$

```
>> pretty(x^7+y^(exp(x/2)))
```

$$y^{\frac{\exp(x/2)}{2}} + x^7$$

ezplot - creates 2-D plot in the x-range from -2π to 2π

```
>> ezplot('3*x*cos(x)')
```



INTEGRATION

trapz: Estimates an integral using the trapezoidal rule

- Trapezoidal rule:

$$\int_{x_0}^{x_1} f(x)dx = \frac{h}{2} [f(x_0) + f(x_1)]$$

where $h = (x_1 - x_0)$

- Usage:

trapz(x,y)

Limits of
integration in
vector form

Function to be
integrated – can use
anonymous function

INTEGRATION

Trapezoidal Rule Example:

- Estimate the integral shown

$$\int_0^{0.5} \frac{2}{x-4} dx$$

```
Command Window
>> f=@(x) 2./(x-4);
>> x=[0 0.5];
>> trapz(x,f(x))

ans =

-0.267857142857143
```

INTEGRATION

quad: implements Simpson's method for approximating integrals

- More accurate than trapezoidal rule
- Simpson's Method:

$$\int_{x_0}^{x_2} f(x) dx = \frac{h}{3} [f(x_0) + 4f(x_1) + f(x_2)]$$

where $h = (x_2 - x_0)$

$$x_1 = x_0 + h$$

- Usage:

`quad(f, x0, x2)`

Function to
be integrated

INTEGRATION

quad Example:

- Estimate the integral shown

$$\int_0^{0.5} \frac{2}{x-4} dx$$

```
Command Window
>> f=@(x) 2./(x-4);
>> quad(f,0,0.5)

ans =

-0.267062785251494
```

POLYNOMIAL CALCULUS

- **polyint**: returns the integral of a polynomial
 - Usage: `polyint(c)`
 - `c`=row vector polynomial coefficients
 - Example: Calculate the integral below

$$\int 4x^2 + 6x + 7 \, dx$$

```
Command Window
>> coeffs=[4 6 7];
>> int = polyint(coeffs)

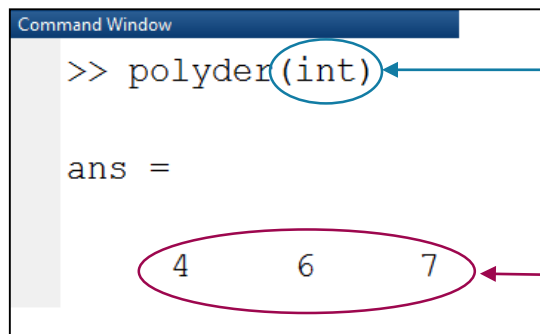
int =
    1.3333    3.0000    7.0000    0
```

$$1.3333x^3 + 3x^2 + 7x$$

POLYNOMIAL CALCULUS

- **polyder**: returns the derivative of a polynomial
 - Usage: `polyder(c)`
 - `c`=row vector of polynomial coefficients
 - Example: Find the derivative of

$$1.3333x^3 + 3x^2 + 7x$$



```
Command Window
>> polyder(int)
ans =
    4    6    7
```

`int=[1.3333 3 7 0]`

$$4x^2 + 6x + 7$$

SYMBOLIC CALCULUS

- **diff**: differentiates a symbolic expression
 - Returns a symbolic expression
 - Usage: `diff(f)`
 - `f`=symbolic expression
 - Example: find the symbolic derivative of

$$1.3333x^3 + 3x^2 + 7x$$

```
Command Window
>> syms x
>> f=1.333*x^3+3*x^2+7*x;
>> diff(f)

ans =

(3999*x^2)/1000 + 6*x + 7
```

Remember: Symbolic expressions do not use floating point numbers

SYMBOLIC CALCULUS

- **int**: integrates symbolic expression
 - Returns a symbolic expression
 - Works with definite and indefinite integrals

$$\int_a^b f(x) dx \qquad \int f(x) dx$$

- Usage: `int(f, a, b)` `int(f)`
- Example: Integrate the following

$$\int 2 \sin(x) + 1 dx$$

$$\int_{-2\pi}^{2\pi} 2 \sin(x) + 1 dx$$

f is symbolic

```

Command Window
>> f=2*sin(x)+1;
>> int(f)

ans =

x - 2*cos(x)

>> int(f,-2*pi,2*pi)

ans =

4*pi
  
```

STATISTICS

- **mean:** returns the average of numbers

$$\frac{\sum_{i=1}^n x_i}{n}$$

- Usage: mean(x)
 - x=vector of numbers
- Example: find the average of the following numbers
1, 5, 7, 8, 3, 2, 24,21

```
x=[1 5 7 8 3 2 24 21];
```

```
mean(x)
```

```
ans =
```

```
8.8750
```

STATISTICS

- **var**: calculates the *variance* of data
 - Gives measure of spread of data
 - Higher variance indicates more variation (less consistency) in the data points

$$variance = \frac{\sum_{i=1}^n (x_i - mean)^2}{n - 1}$$

- Usage: var(x)
 - x=vector of data points
- **std**: calculates the *standard deviation* of data

$$std = \sqrt{variance}$$

- Usage: std(x)
 - x=vector of data points

STATISTICS

- Example: Find the standard deviation and variance of the following numbers

1, 5, 7, 8, 3, 2, 24, 21

```
x=[1 5 7 8 3 2 24 21;
```

```
var(x)
```

```
>>76.9821
```

```
std(x)
```

```
>>8.7739
```

STATISTICS

- **mode**: returns number that appears most frequently in data
 - Chooses the smallest mode if there are multiple possible modes
- **median**: returns middle number of data
 - For even length data sets, two middle numbers are averaged

```
x=[1 5 5 5 3 2];
```

```
mode(x)
```

```
median(x)
```

```
>>mode=5
```

```
>>median=4
```